

Non-Recursive Algorithm Analysis

1. Algorithm Description:

The algorithm first sorts the input array in ascending order. After sorting, it checks every consecutive triplet ($A[i]$, $A[i+1]$, $A[i+2]$) to determine if they satisfy the triangle condition:

$$A[i] + A[i+1] > A[i+2]$$

If any valid triplet is found, the function returns 1, otherwise it returns 0.

2. Time Complexity Analysis:

- Sorting the array takes: $O(n \log n)$
- The loop that checks triplets runs: $O(n)$

3. Final Complexity:

$O(n \log n)$

Because sorting dominates the overall complexity.

4. Correctness Explanation:

After sorting:

$$A[i] \leq A[i+1] \leq A[i+2]$$

So we only need to check:

$$A[i] + A[i+1] > A[i+2]$$

If this condition is true for any index i , a triangular triplet exists.

5. Conclusion:

The non-recursive approach is efficient because it reduces the problem from checking all triplets ($O(n^3)$) to a single pass after sorting ($O(n \log n)$). It is simple, fast, and optimal for this problem.